

Universidad de Guayaquil
Facultad de Ingeniería Industrial
Carrera de Ingeniería en Telemática

Materia:

Ingeniería de Software

Tema:

Etapas 3: Modelado Del Sistema y Diseño de la Arquitectura, Proyecto:
Desarrollo y Análisis de Inteligencias Artificiales en la Creación del
Videojuego "El Último Bastión"

Grupo de Trabajo:

Leon Briones Juan Elias

Rodriguez Calderon Jordan Josue

Intriago Celi Bladimir Sebastian

Vicuña Aspiazu Mario Samuel

Docente:

Ing. Ángel Plaza Vargas, MSC.

Fecha de Entrega:

5 de junio de 2026

Año Lectivo:

2025 – 2026 CI

Contenido

1. Introducción.....	4
2. Objetivos.....	4
2.1. Objetivo General.....	4
2.2. Objetivos Específicos.....	4
3. Descripción General del Proyecto.....	4
4. Modelo de Contexto del Sistema.....	5
4.1. Descripción.....	5
4.2. Diagrama de Contexto.....	5
4.3. Análisis del Modelo de Contexto.....	5
5. Modelo de Casos de Uso.....	6
5.1. Identificación de Actores.....	6
5.2. Casos de Uso Identificados.....	6
5.3. Diagrama General de Casos de Uso.....	6
5.4. Diagrama de Casos de Uso del Sistema de Combate.....	7
6. Especificaciones de casos de Uso.....	8
6.1. Caso de Uso CU-01: Iniciar Partida.....	8
Precondiciones.....	8
Flujo Principal.....	9
Flujo Alternativo.....	9
Postcondiciones.....	9
6.2. Caso de Uso CU-02: Construir Torre.....	9
Precondiciones.....	9

Flujo Principal.....	9
Flujo Alternativo.....	9
Postcondiciones.....	10
7. Diagramas de Secuencia.....	10
7.1. Diagrama de Secuencia: Construcción de Torre.....	10
7.2. Diagrama de Secuencia: Ataque de Torre a Enemigos.....	10
8. Diseño de la Arquitectura.....	11
8.1. Género Arquitectónico.....	11
8.2. Estilo Arquitectónico.....	11
8.3. Arquitectura Propuesta.....	12
8.4. Diagrama Arquitectónico.....	13
9. Análisis de la Arquitectura Propuesta.....	13
10. Conclusión.....	13

1. Introducción

El presente documento corresponde a la etapa de Modelado del Sistema y Diseño de la Arquitectura del proyecto denominado "El Último Bastión", un videojuego del género Tower Defense desarrollado en Python utilizando la biblioteca Pygame.

El objetivo de esta etapa es representar formalmente la estructura funcional del sistema, identificar los actores que interactúan con él, definir los casos de uso principales y establecer la arquitectura de software que servirá como base para el desarrollo posterior del videojuego.

Mediante la aplicación de técnicas de modelado UML y principios de arquitectura de software se busca obtener una visión clara del comportamiento y organización interna del sistema antes de iniciar las fases avanzadas de implementación.

2. Objetivos

2.1. Objetivo General

Modelar y diseñar la arquitectura del videojuego "El Último Bastión" utilizando técnicas de Ingeniería de Software para establecer una base sólida para su desarrollo.

2.2. Objetivos Específicos

- Desarrollar el modelado del sistema, definiendo la estructura funcional y las interacciones necesarias para el cumplimiento de los requerimientos del proyecto.
- Establecer la representación del comportamiento del sistema, describiendo los procesos fundamentales que intervienen en la ejecución de sus funcionalidades principales.
- Diseñar la arquitectura del software, determinando la organización de sus componentes y los mecanismos que permitan su correcto funcionamiento e integración.

3. Descripción General del Proyecto

El Último Bastión es un videojuego de estrategia tipo Tower Defense en el que el jugador debe proteger una fortaleza frente a oleadas sucesivas de enemigos.

El jugador podrá construir torres defensivas, administrar recursos, mejorar estructuras y evitar que los enemigos destruyan el bastión principal.

El sistema será desarrollado utilizando Python y Pygame, siguiendo principios de programación orientada a objetos y una arquitectura modular.

4. Modelo de Contexto del Sistema

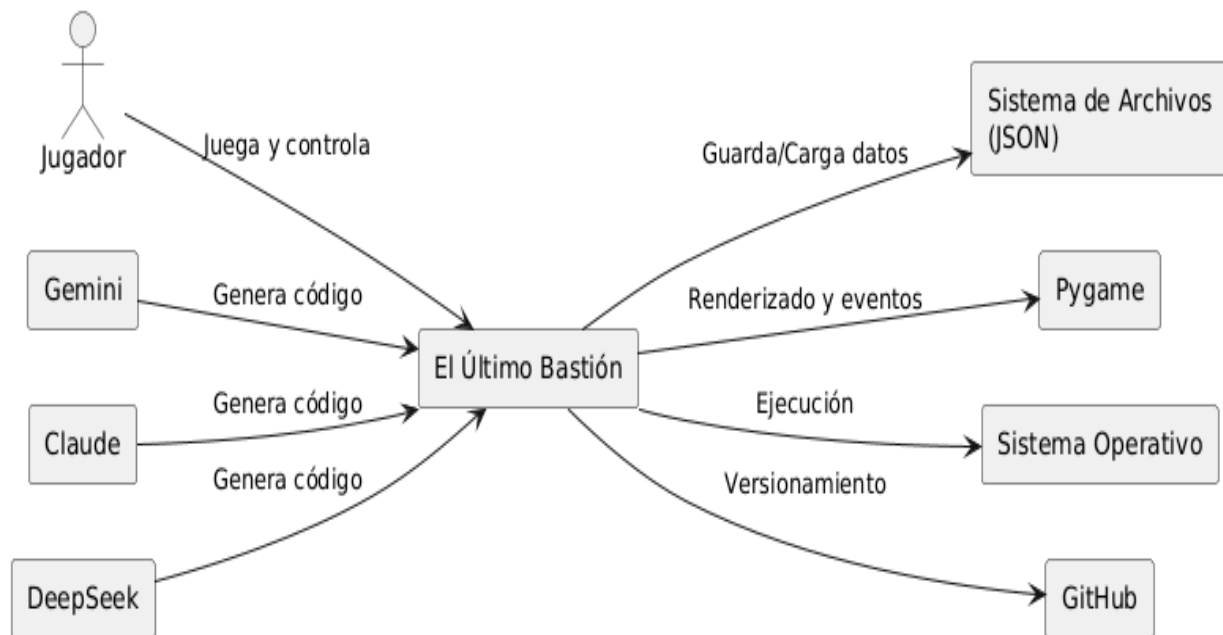
4.1. Descripción

El modelo de contexto permite identificar los límites del sistema y los elementos externos que interactúan con él.

Los principales actores y sistemas externos son:

- Jugador
- Motor Pygame
- Sistema Operativo
- Sistema de Archivos
- GitHub
- Inteligencias Artificiales utilizadas durante el desarrollo

4.2. Diagrama de Contexto



4.3. Análisis del Modelo de Contexto

El jugador constituye el actor principal del sistema debido a que interactúa directamente con las funcionalidades del videojuego. Los sistemas externos permiten la ejecución, almacenamiento de información, control de versiones y apoyo al desarrollo mediante inteligencia artificial.

5. Modelo de Casos de Uso

5.1. Identificación de Actores

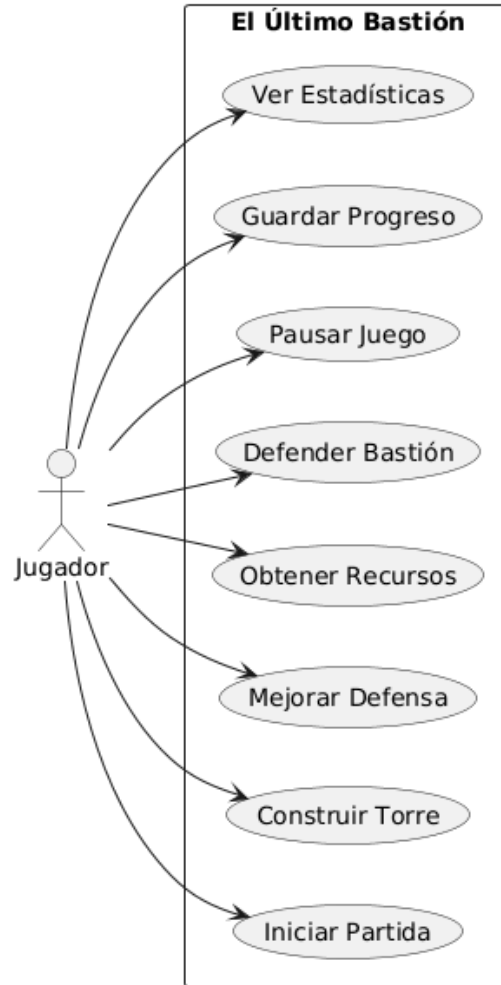
Actor Principal: Jugador

Actores Secundarios: Sistema de Archivos y Motor Pygame

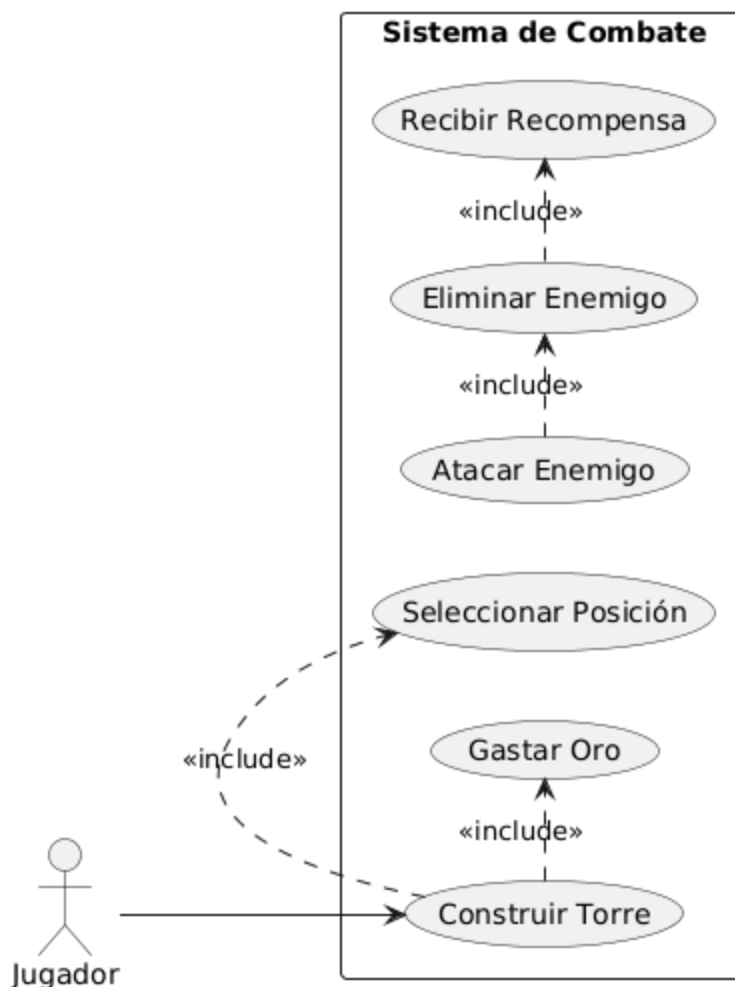
5.2. Casos de Uso Identificados

- Iniciar Partida
- Construir Torre
- Mejorar Defensa
- Obtener Recursos
- Defender Bastión
- Pausar Juego
- Guardar Progreso
- Ver Estadísticas

5.3. Diagrama General de Casos de Uso



5.4. Diagrama de Casos de Uso del Sistema de Combate



6. Especificaciones de casos de Uso

6.1. Caso de Uso CU-01: Iniciar Partida

Nombre:	Iniciar Partida
Actor Principal	Jugador
Descripción:	Permite al usuario comenzar una nueva sesión de juego.
Precondiciones	El videojuego se encuentra ejecutándose correctamente.

Flujo Principal	1. El jugador accede al menú principal. 2. Selecciona la opción Nueva Partida. 3. El sistema carga el escenario inicial. 4. Se inicializan los recursos del jugador. 5. Comienza la primera oleada.
Flujo Alternativo	Error durante la carga del escenario.
Postcondiciones	Partida iniciada correctamente.

6.2. Caso de Uso CU-02: Construir Torre

Nombre:	Construir Torre
Actor Principal	Jugador
Descripción:	Permite colocar una torre defensiva dentro del escenario utilizando recursos disponibles.
Precondiciones	El jugador posee suficiente oro. Existe una posición válida para construcción.
Flujo Principal	1. El jugador selecciona una torre. 2. Selecciona una posición del mapa. 3. El sistema verifica los recursos. 4. Se construye la torre. 5. El sistema descuenta el costo correspondiente.
Flujo Alternativo	Recursos insuficientes.

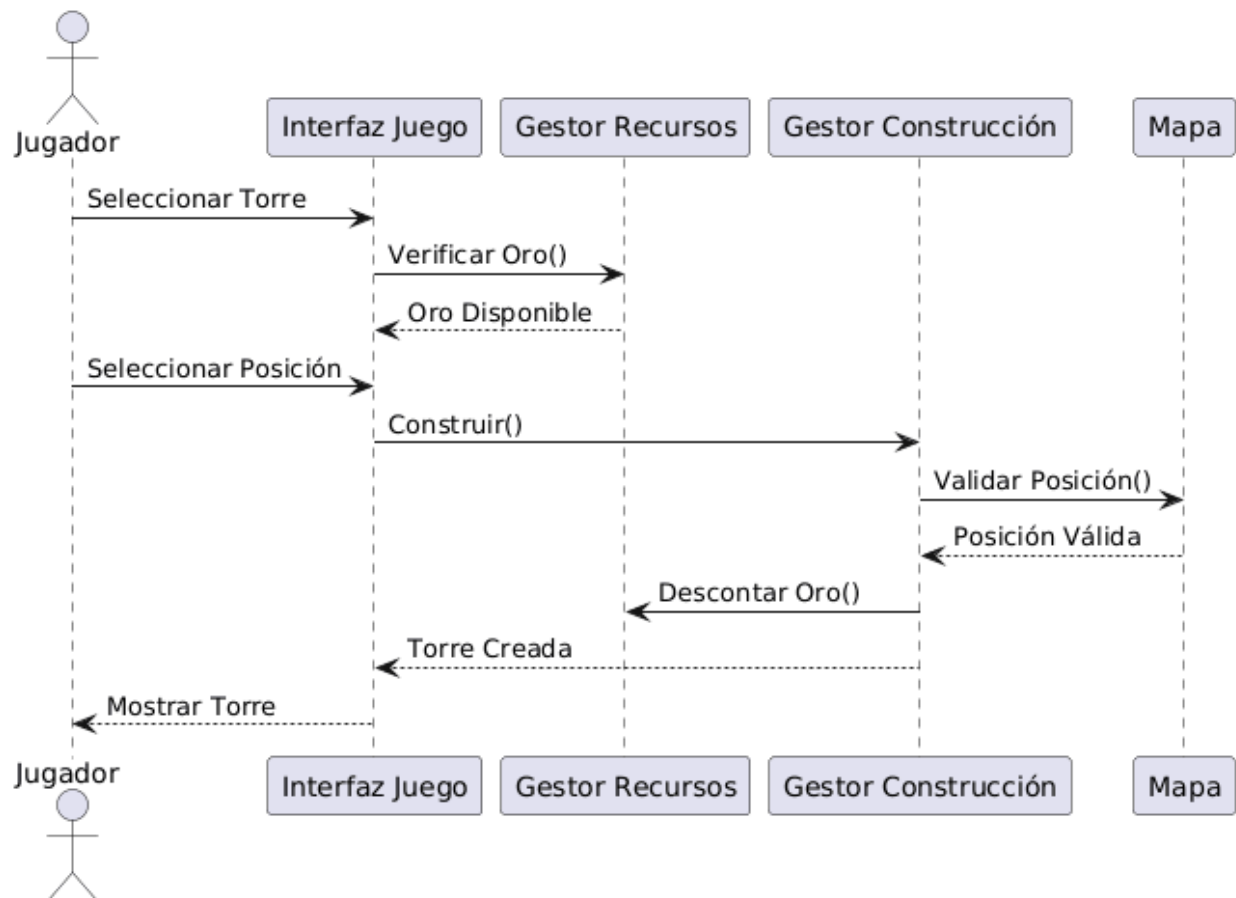
Postcondiciones	Torre creada exitosamente.
------------------------	----------------------------

7. Diagramas de Secuencia

Los diagramas de secuencia permiten representar el flujo temporal de mensajes intercambiados entre los objetos participantes de un proceso.

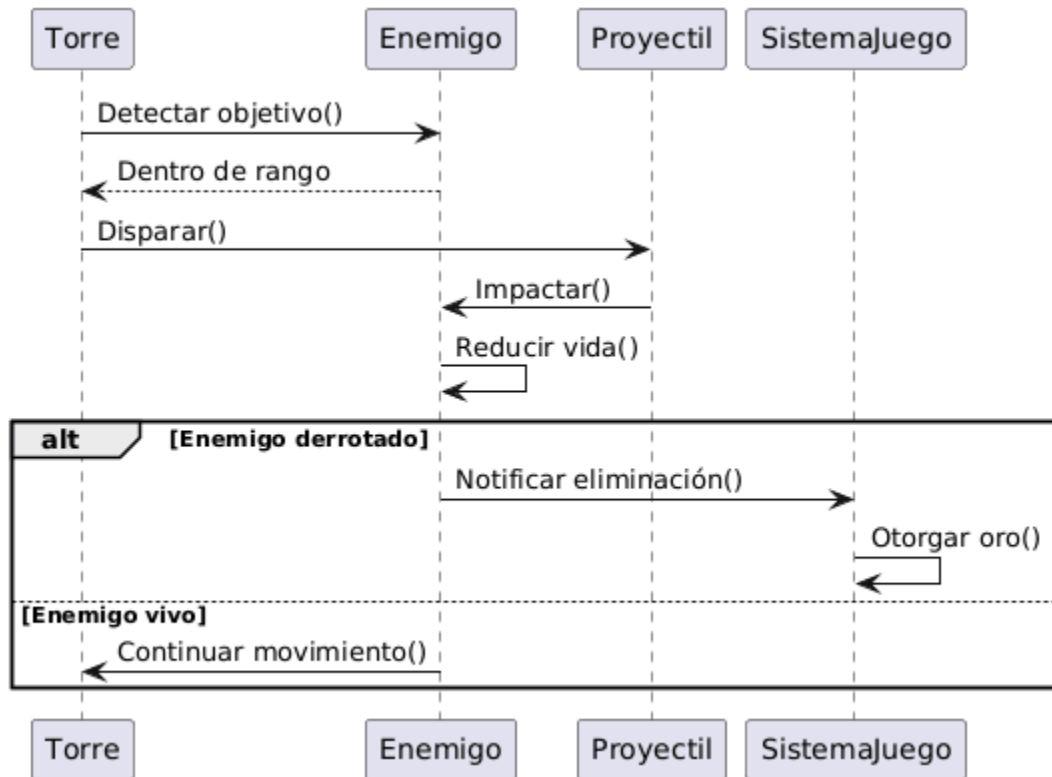
7.1. Diagrama de Secuencia: Construcción de Torre

Representa la interacción entre el jugador, la interfaz de usuario y los módulos internos durante la construcción de una torre.



7.2. Diagrama de Secuencia: Ataque de Torre a Enemigos

Representa el proceso de detección, ataque y eliminación de enemigos por parte de una torre defensiva.



8. Diseño de la Arquitectura

8.1. Género Arquitectónico

De acuerdo con la clasificación de géneros arquitectónicos, el proyecto pertenece al género: Juegos (Games). Esto se debe a que el sistema tiene como objetivo proporcionar una experiencia interactiva de entretenimiento mediante mecánicas de juego en tiempo real.

8.2. Estilo Arquitectónico

El estilo arquitectónico seleccionado es: Arquitectura Orientada a Objetos

La elección se fundamenta en:

- Modularidad.
- Reutilización de código.

- Encapsulamiento.
- Facilidad de mantenimiento.
- Compatibilidad con Python.

Adicionalmente se implementará una organización en capas para separar responsabilidades.

8.3. Arquitectura Propuesta

El sistema estará dividido en las siguientes capas:

Capa de Presentación

Responsable de:

- Menús.
- Pantallas.
- HUD.
- Interacción visual.

Capa de Lógica de Negocio

Responsable de:

- Control del juego.
- Sistema de combate.
- Gestión de oleadas.
- Construcción de torres.

Capa de Entidades

Responsable de:

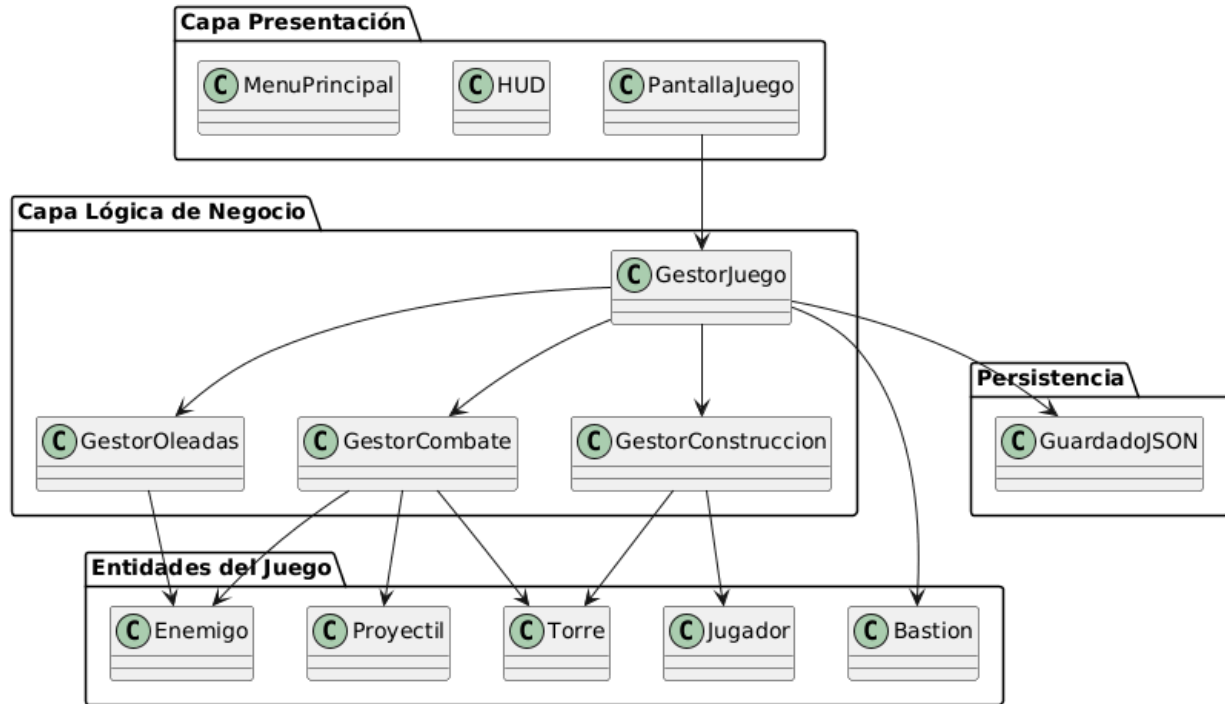
- Jugador.
- Torres.
- Enemigos.
- proyectiles.
- Bastión.

Capa de Persistencia

Responsable de:

- Guardado de partidas.
- Carga de configuraciones.
- Gestión de archivos JSON.

8.4. Diagrama Arquitectónico



9. Análisis de la Arquitectura Propuesta

La arquitectura seleccionada proporciona una estructura modular y escalable que facilita la incorporación de nuevas funcionalidades durante las etapas posteriores del proyecto.

La separación en capas permite mantener independencia entre la interfaz gráfica, la lógica del juego y los mecanismos de almacenamiento de datos.

Además, la utilización de programación orientada a objetos favorece la reutilización de componentes y simplifica las actividades de mantenimiento y evolución del software.

10. Conclusión

El modelado realizado permitió identificar de manera clara los actores y las funcionalidades principales del videojuego, facilitando así la comprensión de los requerimientos del sistema. Asimismo, los diagramas de casos de uso contribuyeron a representar los requerimientos funcionales, mientras que los diagramas de secuencia permitieron visualizar la interacción entre los distintos componentes internos. A partir del análisis efectuado, se determinó que el género arquitectónico del proyecto corresponde a un sistema de juegos interactivos, siendo la arquitectura orientada a objetos la alternativa más adecuada para su desarrollo debido a su modularidad, escalabilidad y facilidad de mantenimiento. En consecuencia, la arquitectura propuesta constituye una base sólida para las futuras etapas de implementación, integración y pruebas del proyecto.

Referencias

Winters, T. & Manshreck, T. (2022). *Ingeniería de software en Google: Lecciones sobre programación aprendidas a lo largo del tiempo*: (1 ed.). Marcombo.

<https://elibro.net/es/lc/uguayaquil/titulos/281863>

Mejía Trejo, J. (2024). *Principios de aseguramiento de calidad para el diseño de software: innovación de procesos en las tecnologías de información*: (1 ed.). Academia Mexicana de Investigación y Docencia en Innovación (AMIDI).

<https://elibro.net/es/lc/uguayaquil/titulos/250101>